MOD -03 -> Advanced Java

LUMINAR
TECHNOLAB

Java Thread

LUMINAR
TECHNOLAB

Java Thread

Java Process

1. A Java process:-> is a program in execution.
2. A Java process consists of multiple threads
3. Java runtime environment runs as a single process which contains different classes and programs as processes.

4. **Java Thread**
5. Java thread is a subset of a Java process.
6. Thread shares the process resources.

1. Every java application has at least one thread - main thread.
2. we can create multiple threads from it.
3. there are so many other java threads running in background like memory management, system management, signal processing etc.
4. Multithreading refers to two or more threads executing concurrently in a single program.
5. Java process has its own address space, a Java thread uses the process' address space and shares it with other threads of that process.

Java provides two ways to create a thread programmatically.

1. Extending the java.lang.Thread class.
2. Implementing the java.lang.Runnable interface.

Class constructors

Sl.No.	Constructor & Description
1	**Thread()** This allocates a new Thread object.
2	**Thread(Runnable target)** This allocates a new Thread object.
3	**Thread(Runnable target, String name)** This allocates a new Thread object.
4	**Thread(String name)** This constructs allocates a new Thread object.
5	**Thread(ThreadGroup group, Runnable target)** This allocates a new Thread object.

6	**Thread(ThreadGroup group, Runnable target, String name)** This allocates a new Thread object so that it has target as its run object, has the specified name as its name, and belongs to the thread group referred to by group.
7	**Thread(ThreadGroup group, Runnable target, String name, long stackSize)** This allocates a new Thread object so that it has target as its run object, has the specified name as its name, belongs to the thread group referred to by group, and has the specified stack size.
8	**Thread(ThreadGroup group, String name)** This allocates a new Thread object.

Java Thread class

Step 1: You will need to override run( ) method available in Thread class. This method provides an entry point for the thread and you will put your complete business logic inside this method.

public void run( )

Step 2: Once Thread object is created, you can start it by calling start() method, which executes a call to run( ) method.

void start( );

```
public class ThreadDemo extends Thread {

    public void run() {
        System.out.println("This code is running in a thread");
    }

  public static void main(String[] args) {
        ThreadDemo thread = new ThreadDemo();
        thread.start();
        System.out.println("This code is outside of the thread");
  }
}
```

Java Runnable interface

Step 1: You will need to override run( ) method available in Runnable interface. This method provides an entry point for the thread and you will put your complete business logic inside this method.

public void run( )

Step 2: instantiate a Thread object using the constructor

Step 3: Once Thread object is created, you can start it by calling start() method, which executes a call to run( ) method.

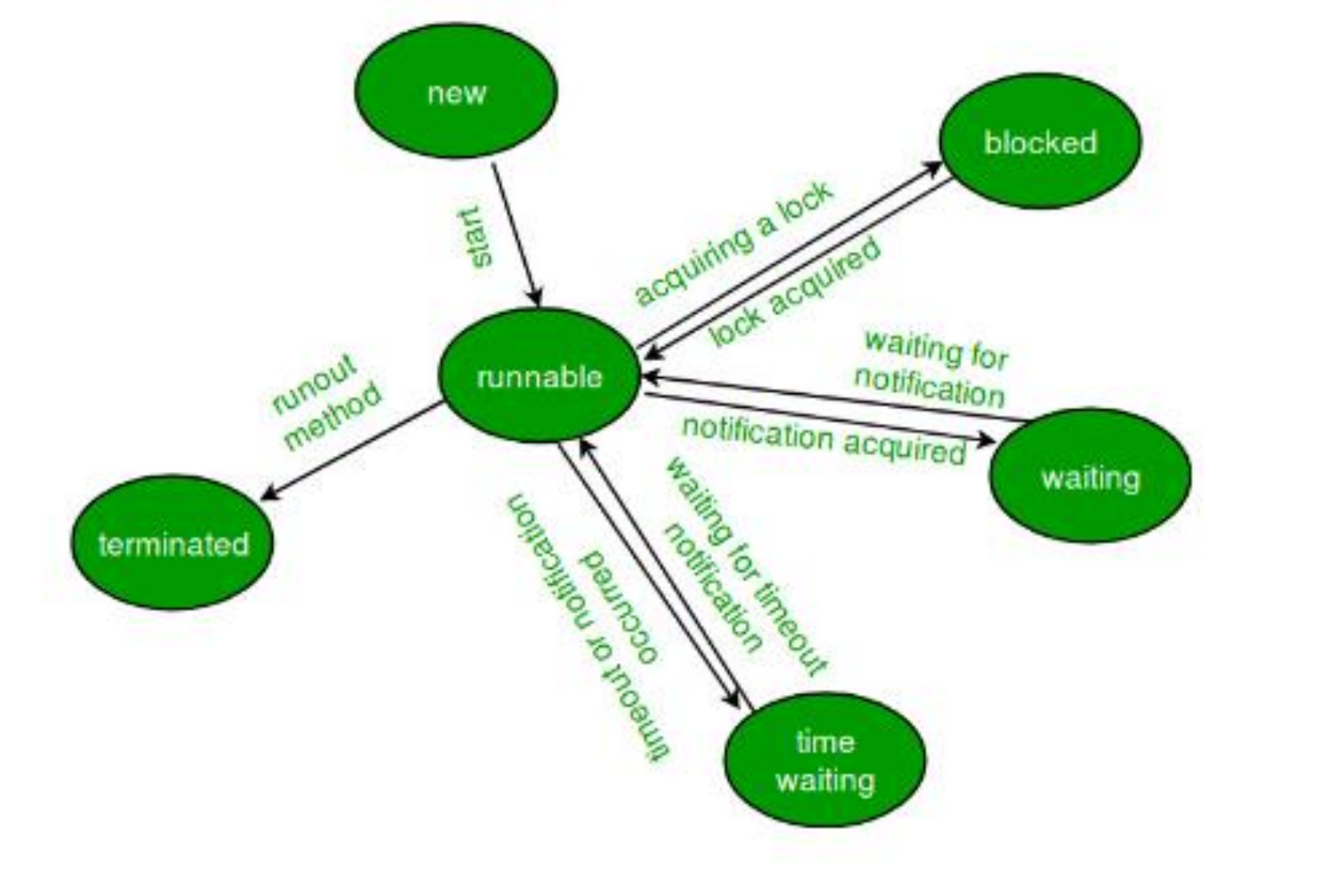
void start( );

```
public class ThreadDemo implements Runnable {

    public void run() {
        System.out.println("This code is running in a thread");
    }

  public static void main(String[] args) {
        ThreadDemo threads = new ThreadDemo();
        Thread thread = new Thread(threads);
          thread.start();
        System.out.println("This code is outside of the thread");
  }
}
```

Lifecycle of a Thread in Java



1. New Thread: When a new thread is created, it is in the new state. The thread has not yet started to run when the thread is in this state. When a thread lies in the new state, its code is yet to be run and hasn't started to execute.

2. Runnable State: A thread that is ready to run is moved to a runnable state. In this state, a thread might actually be running or it might be ready to run at any instant of time. It is the responsibility of the thread scheduler to give the thread, time to run.

A multi-threaded program allocates a fixed amount of time to each individual thread. Each and every thread runs for a short while and then pauses and relinquishes the CPU to another thread so that other threads can get a chance to run. When this happens, all such threads that are ready to run, waiting for the CPU and the currently running thread lie in a runnable state.

3. Blocked/Waiting state: When a thread is temporarily inactive, then it's in one of the following states:
    - Blocked
    - Waiting
4. Timed Waiting: A thread lies in a timed waiting state when it calls a method with a time-out parameter. A thread lies in this state until the timeout is completed or until a notification is received. For example, when a thread calls sleep or a conditional wait, it is moved to a timed waiting state.
5. Terminated State: A thread terminates because of either of the following reasons:

    Because it exits normally. This happens when the code of the thread has been entirely executed by the program.

    Because there occurred some unusual erroneous event, like segmentation fault or an unhandled exception.

Recap - Lifecycle of a Thread in Java

The Life Cycle of a Thread in Java refers to the state transformations of a thread that begins with its birth and ends with its death. In Java, these thread state transformations are referred to as the Thread life cycle.

1. When a thread instance is generated and executed by calling the start() method of the Thread class, the thread enters the runnable state.
2. When the sleep() or wait() methods of the Thread class are called, the thread enters a non-runnable mode.
3. The thread dies when it exits the run() process.

Methods are invoked on a particular Thread object.

#	Method & Description
1	public void start() Starts the thread in a separate path of execution, then invokes the run() method on this Thread object.
2	public void run() If this Thread object was instantiated using a separate Runnable target, the run() method is invoked on that Runnable object.
3	public final void setName(String name) Changes the name of the Thread object. There is also a getName() method for retrieving the name.
4	public final void setPriority(int priority) Sets the priority of this Thread object. The possible values are between 1 and 10.

5	public final void setDaemon(boolean on) A parameter of true denotes this Thread as a daemon thread.
6	public final void join(long millisec) The current thread invokes this method on a second thread, causing the current thread to block until the second thread terminates or the specified number of milliseconds passes.
7	public void interrupt() Interrupts this thread, causing it to continue execution if it was blocked for any reason.
8	public final boolean isAlive() Returns true if the thread is alive, which is any time after the thread has been started but before it runs to completion.

Invoking one of the static methods performs the operation on the currently running thread.

public static void sleep(long millisec)

Causes the currently running thread to block for at least the specified number of milliseconds.

public static Thread currentThread()

Returns a reference to the currently running thread, which is the thread that invokes this method.

```java
package threaddemos;


public class TestSleepMethod extends Thread{

    public void run(){
        for(int i=1;i<5;i++){
            try{
               Thread.sleep(500); // the thread will sleep for the 500 milli seconds
            }catch(InterruptedException e){
                e.printStackTrace();
            }
        System.out.println(i);
        }
    }

    public static void main(String args[]){
     TestSleepMethod t1=new TestSleepMethod();
     TestSleepMethod t2=new TestSleepMethod();

     t1.start();
     t2.start();
    }
}
```

```
<terminated> TestSlee
1
1
2
2
3
3
4
4
```

Practice Questions

Question 1: Write a program to print “Luminar" and “Kochi" continuously on the screen in Java using threads.

```java
package threaddemos;

class ThreadKochi extends Thread{
    public void run(){
        while (true){
        System.out.println("Kochi");
        }
    }
}

class ThreadLuminar extends Thread {
    public void run() {
        while (true) {
            System.out.println("Luminar");
        }
    }
}

public class LuminarKochiThreadTest {
    public static void main(String[] args) {
        ThreadLuminar luminar= new ThreadLuminar();
        ThreadKochi kochi= new ThreadKochi();
        luminar.start();
        kochi.start();
    }
```

Priority of a Thread (Thread Priority)

Each thread has a priority. Priorities are represented by a number between 1 and 10. In most cases, the thread scheduler schedules the threads according to their priority (known as preemptive scheduling). But it is not guaranteed because it depends on JVM specification that which scheduling it chooses. Note that not only JVM a Java programmer can also assign the priorities of a thread explicitly in a Java program

Setter & Getter Method of Thread Priority

public final int getPriority(): The java.lang.Thread.getPriority() method returns the priority of the given thread.

public final void setPriority(int newPriority): The java.lang.Thread.setPriority() method updates or assign the priority of the thread to newPriority.

The method throws IllegalArgumentException if the value newPriority goes out of the range, which is 1 (minimum) to 10 (maximum).

3 constants defined in Thread class(Thread Priorities)

public static int MIN_PRIORITY

public static int NORM_PRIORITY

public static int MAX_PRIORITY

- Default priority of a thread is 5 (NORM_PRIORITY).
- The value of MIN_PRIORITY is 1
- The value of MAX_PRIORITY is 10.

```java
package threaddemos;


class ThreadWelcomez extends Thread{
    public void run(){
        System.out.println("Welcome");
    }
}
class ThreadKochiz extends Thread {
    public void run() {
            System.out.println("Kochi");
        }
}
class ThreadLuminarz extends Thread {
    public void run() {
            System.out.println("Luminar");
        }
}
```

```java
public class ThreadPriorityExample {
    public static void main(String[] args) {

            ThreadWelcomez t1= new ThreadWelcomez();
            ThreadKochiz t2= new ThreadKochiz();
            ThreadLuminarz t3=new ThreadLuminarz();

            t1.start();
            t2.start();
            t3.start();

            t1.setPriority(5);
            t2.setPriority(1); //t2.setPriority(Thread.MIN_PRIORITY);
            t3.setPriority(10);

            System.out.println(t1.getPriority());
            System.out.println(t2.getPriority());
            System.out.println(t3.getPriority());

            }
}
```

Console

<terminated> ThreadPriorityExa

```
5
1
10
Luminar
Welcome
Kochi
```

Thank you ☺ Happy learning ☺